Como tudo se conecta - Semana 5

Formulários, rotas, pipes, diretivas e APIs no projeto biblioteca-web

Desenvolvimento Front-End [Angular] T1 | Módulo 2 | Semana 5 | Mapa visual da semana: como formulários, rotas, pipes, diretivas e APIs se conectam no projeto **biblioteca-web**. Use como cola durante os exercícios.

---

1. Visão geral: como a biblioteca-web funciona agora

Usuário
digita, clica e navega

→

Router
`provideRouter`: URL → página (guards, lazy)

→

Páginas (componentes)
Lista, Detalhe, Cadastro de livro, Cadastro de leitor, Endereço, Catálogo

→

Services
`LivroService` (signals), `CepService`, `CatalogoService`

→

APIs externas
viaCEP · Open Library · RapidAPI (modelo)

Na tela: **pipes** formatam (`currency`, `date`, `tempoLeitura`) e **diretivas** reutilizam comportamento (`appDestaque`, `appSomenteNumeros`). Entre os services e as APIs passa o **interceptor** (log e cabeçalhos).

2. Formulários: duas abordagens para o mesmo problema

Template-Driven (leitor)
`FormsModule` · `[(ngModel)]` + `name` · validadores em **atributos** (`required`, `minlength`, `email`) · `#f="ngForm"` · `(ngSubmit)="salvar(f)"`

Reactive Forms (livro)
`ReactiveFormsModule` · `NonNullableFormBuilder.group({...})` · `Validators` e **validador personalizado** · `[formGroup]` + `formControlName` · `(ngSubmit)="enviar()"`

Estados do campo
`valid` / `invalid` · `touched` / `untouched` · `dirty` / `pristine` → classes `ng-*`

→

Mostrar o erro
`invalid` **e** `touched` (ou após o envio, com `markAllAsTouched()`)

→

Enviar
se válido: `LivroService.adicionar()` com **lista nova** (`update`) e `reset()`

3. Rotas: da URL ao componente

URL
`/livros/3`

Router

Guard

Componente

→ procura a **primeira** rota que combina (`**` por último) → `canDeactivate` : pode sair? (`form.dirty`) → `DetalheLivro` com `id = input.required<string>()`

→ **router-outlet**
mostra a página sem recarregar

routerLink
o usuário **clica**: `[routerLink]="['/livros', id]"` →

router.navigate
depois de uma **ação** no código: `navigate(['/livros', novo.id])` →

Lazy loading
`loadComponent: () => import(...)` : o código só baixa ao entrar

4. Pipes e diretivas: formatar e reutilizar

Pipe: só na tela
`{{ 59.9 | currency:'BRL' }}` → R$ 59,90 · `date` , `uppercase` , `percent` , `slice` , `json` , `async` · personalizado: `@Pipe` + `transform()` · idioma: `registerLocaleData` + `LOCALE_ID`

Diretiva de atributo
`@Directive` + `host` (eventos) + `inject(ElementRef)` + `input()` com alias · `appDestaque` , `appSomenteNumeros`

Os 3 tipos de diretiva
componentes (com template) · **estruturais** (`@if` , `@for`) · **de atributo** (`ngClass` , as nossas)

5. APIs: do clique ao dado na tela

Componente
chama o service e controla os **3 estados** com signals →

Service
`inject(HttpClient)` · `get<T>(url, { params })` →

Interceptor
`withInterceptors([...])` : log, cabeçalhos (`req.clone()`) →

Internet
viaCEP / Open Library (**CORS** liberado pelo servidor)

→ **Resposta**
`map` para o nosso modelo → `subscribe` → signals

Carregando
`carregando.set(true)` antes da chamada →

Sucesso
`next` : guarda o resultado / `patchValue` →

Erro
`error` : mensagem útil (`status` 0, 404, 500) →

Sempre
`finalize` : `carregando.set(false)`

CEP digitado
`appSomenteNumeros` + `maxlength="8"` →

valueChanges →

CepService.buscar →

patchValue
preenche rua, bairro, cidade e UF

filter(length === 8) + takeUntilDestroyed()

viaCEP: `{ "erro": true }` = CEP inexistente (**200**!)

6. Qual ferramenta usar? Guia de decisão

Pergunta	Use	Por quê
Formulário simples (3 a 5 campos)?	**Template-Driven**	Validação em atributos, pouco código
Formulário com muitos campos, regras entre campos ou dinâmico?	**Reactive Forms**	Lógica na classe: organizada e testável
Preciso de várias páginas com endereço próprio?	**Router** (`routes`)	SPA sem recarregar, URL compartilhável
Quero impedir a saída de uma página com dados não salvos?	**Guard** `canDeactivate`	Função que libera ou bloqueia
Quero só mudar **como o valor aparece**?	**Pipe**	Não altera o dado original
Quero reaproveitar um **comportamento** em vários elementos?	**Diretiva de atributo**	Escuta eventos e muda o elemento
Preciso de dados de um servidor?	`HttpClient` **em um service**	Observable + 3 estados com signals
Quero a mesma regra em **toda** requisição (log, token)?	**Interceptor**	Um ponto central
A API pede uma chave?	**Back-end próprio** (produção)	No front a chave fica exposta no DevTools